ALScreen Framework Pseudocode

ScoreFormer.Train

Input: $D = \{(m_i, y_i)\}$: molecules m_i and their docking scores y_i;
H: hyperparameters (optimizer, learning-rate schedule, batch size, weighting strength α, log-weight cap c, early-stopping patience p);
E: maximum number of training epochs.

Output: f_θ: trained ScoreFormer.

$G \leftarrow \{(G_i, y_i)\}$, with $G_i = \text{Graph}(m_i)$, for each $(m_i, y_i) \in D$.
Split G into a training set G_{tr} and a validation set G_{val}.
Initialize θ, the optimizer, and the learning-rate schedule from H.
$\ell^* \leftarrow +\infty$; $\theta^* \leftarrow \theta$; $s \leftarrow 0$.
for e = 1, …, E **do**
    Set f_θ to training mode.
    for each mini-batch $B \subseteq G_{tr}$ **do**
        $\hat{y}_i \leftarrow f_\theta(G_i)$ for each $i \in B$.
        $w_i \leftarrow \exp(\min(-\alpha \cdot y_i, c))$ for each $i \in B$. // more negative (better) scores get larger weights
        $L_B \leftarrow (1 / |B|) \cdot \sum_{i \in B} w_i \cdot (\hat{y}_i - y_i)^2$.
        Update θ using the gradient of L_B and the optimizer.
    end for
    Set f_θ to evaluation mode.
    Compute validation loss and prediction metrics on G_{val}. // monitoring only
    $\theta^{(e)} \leftarrow \text{copy}(\theta)$; save checkpoint $\theta^{(e)}$.
    Step the learning-rate schedule.
end for
return $f_\theta^{(E)}$.

ScoreFormer.Infer

Input: Q: compounds to be scored;
$f_\theta^{(E)}$: trained model returned by ScoreFormer.Train.

Output: $\hat{Y} = \{(m, \hat{y}_m)\}$: predicted docking scores keyed by compound identifier.

Set $f_\theta^{(E)}$ to evaluation mode (dropout off, gradients disabled).
$\hat{Y} \leftarrow \varnothing$.
for each mini-batch $B \subseteq Q$ **do**
    $G_B \leftarrow \{\text{Graph}(m) : m \in B\}$.
    $\hat{y}_B \leftarrow f_\theta^{(E)}(G_B)$.
    $\hat{Y} \leftarrow \hat{Y} \cup \{(m, \hat{y}_m) \text{ for } m \in B\}$.
end for
return $\hat{Y}$.

ALScreen (active-learning virtual screening)

Input: L: compound library;
O_T: docking oracle for target T;
n_{sub}: initial subset size;
n_{oracle}: compounds docked per cycle;
n_{pool}: compounds added to the training pool per cycle;
K: maximum number of cycles;
H, E: training settings.

Output: R: ranked list of prioritized compounds.

$S \leftarrow \text{RepresentativeSubset}(L, n_{sub})$.
$D \leftarrow O_T(S)$. // docking labels $\{(m, y_m)\}$
$P_0 \leftarrow D$; $D_{all} \leftarrow D$. // training pool; all docked compounds
$f^{(0)} \leftarrow \text{ScoreFormer.Train}(P_0, H, E)$. // initial training
$T_0 \leftarrow \varnothing$; $k_{final} \leftarrow 0$.
for $k = 1, \ldots, K$ **do**
  $\hat{Y}_k \leftarrow \text{ScoreFormer.Infer}(L, f^{(k-1)})$.
  $G_k \leftarrow$ the n_{oracle} compounds in L with the lowest $\hat{y}_m$.
  $D_k \leftarrow O_T(G_k)$.
  $D_{all} \leftarrow D_{all} \cup D_k$.
  $e_m \leftarrow |\hat{y}_m - y_m|$ for each $m \in G_k$.
  $C_k \leftarrow$ the n_{pool} best- and n_{pool} worst-predicted compounds of G_k (lowest and highest e_m), with their docking labels.
  $P_k \leftarrow P_{k-1} \cup C_k$.
  $f^{(k)} \leftarrow \text{ScoreFormer.Train}(P_k, H, E)$. // retrained from scratch
  $k_{final} \leftarrow k$.
end for
$\hat{Y} \leftarrow \text{ScoreFormer.Infer}(L, f^{(k_{final})})$.
$s_m \leftarrow y_m$ if $m \in D_{all}$, otherwise $\hat{y}_m$, for each $m \in L$.
$R \leftarrow L$ sorted by ascending s_m.
return R.